

EECS 348 Assignment 2 — GenAI Code Comparison

Student: Hunter Noravong

Assignment: EECS 348 Assignment 2

Date: September 17, 2026

1. GenAI Programs:

GenAI #1

Name: ChatGPT

How accessed: ChatGPT web application

GenAI #2

Name: Claude

How accessed: Claude web application

2. Exact Prompt:

Write a C program for EECS 348 Assignment 2. The program will prioritize emails for a busy company CEO. Use a MaxHeap as a priority queue, implemented from scratch using a list-based implementation. Do not use any pre-existing heap modules. The program should read a test file containing EMAIL commands followed by NEXT, READ, and COUNT commands. Emails have the format EMAIL <sender category>,<subject line>,<date>, where sender categories are Boss, Subordinate, Peer, ImportantPerson, and OtherPerson. Priority order is Boss, Subordinate, Peer, ImportantPerson, OtherPerson. If multiple emails have the same sender category, the newest email should have higher priority. Dates are in MM-DD-YYYY format. NEXT should display the highest-priority unread email without removing it. READ should remove the highest-priority unread email without displaying it. COUNT should display the current number of unread emails. Handle NEXT or READ when the queue is empty and multiple NEXT or READ commands in a row. Provide well-commented C source code.

3. Generated Code — GenAI #1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define INITIAL_CAPACITY 10
```

```
#define MAX_LINE_LENGTH 1024
```

```
#define MAX_SUBJECT_LENGTH 512
```

```
typedef struct {
```

```

    char sender[32];
    char subject[MAX_SUBJECT_LENGTH];
    int month;
    int day;
    int year;
    long sequence;
} Email;

typedef struct {
    Email *items;
    int size;
    int capacity;
} MaxHeap;

int sender_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    return 1;
}

int date_value(const Email *email) {
    return email->year * 10000 + email->month * 100 + email->day;
}

int higher_priority(const Email *a, const Email *b) {
    int priority_a = sender_priority(a->sender);
    int priority_b = sender_priority(b->sender);

    if (priority_a != priority_b) {
        return priority_a > priority_b;
    }

    if (date_value(a) != date_value(b)) {
        return date_value(a) > date_value(b);
    }

    return a->sequence > b->sequence;
}

void heap_init(MaxHeap *heap) {
    heap->items = malloc(INITIAL_CAPACITY * sizeof(Email));

```

```

    if (heap->items == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    heap->size = 0;
    heap->capacity = INITIAL_CAPACITY;
}

void heap_destroy(MaxHeap *heap) {
    free(heap->items);
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

void swap_email(Email *a, Email *b) {
    Email temporary = *a;
    *a = *b;
    *b = temporary;
}

void heap_grow(MaxHeap *heap) {
    int new_capacity = heap->capacity * 2;
    Email *new_items = realloc(
        heap->items,
        new_capacity * sizeof(Email)
    );

    if (new_items == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        heap_destroy(heap);
        exit(EXIT_FAILURE);
    }

    heap->items = new_items;
    heap->capacity = new_capacity;
}

void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        if (!higher_priority(

```

```

        &heap->items[index],
        &heap->items[parent])) {
    break;
}

swap_email(
    &heap->items[index],
    &heap->items[parent]
);

index = parent;
}
}

void heapify_down(MaxHeap *heap, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heap->size &&
            higher_priority(
                &heap->items[left],
                &heap->items[largest])) {
            largest = left;
        }

        if (right < heap->size &&
            higher_priority(
                &heap->items[right],
                &heap->items[largest])) {
            largest = right;
        }

        if (largest == index) {
            break;
        }

        swap_email(
            &heap->items[index],
            &heap->items[largest]
        );

        index = largest;
    }
}

```

```
    }  
}
```

```
void heap_insert(MaxHeap *heap, Email email) {  
    if (heap->size == heap->capacity) {  
        heap_grow(heap);  
    }  
  
    heap->items[heap->size] = email;  
    heapify_up(heap, heap->size);  
    heap->size++;  
}
```

```
Email *heap_peek(MaxHeap *heap) {  
    if (heap->size == 0) {  
        return NULL;  
    }  
  
    return &heap->items[0];  
}
```

```
int heap_remove_max(MaxHeap *heap, Email *removed) {  
    if (heap->size == 0) {  
        return 0;  
    }  
  
    *removed = heap->items[0];  
    heap->size--;  
  
    if (heap->size > 0) {  
        heap->items[0] = heap->items[heap->size];  
        heapify_down(heap, 0);  
    }  
  
    return 1;  
}
```

```
void print_email(const Email *email) {  
    printf("Next email:\n");  
    printf("Sender: %s\n", email->sender);  
    printf("Subject: %s\n", email->subject);  
    printf(  
        "Date: %02d-%02d-%04d\n",  
        email->month,
```

```

        email->day,
        email->year
    );
}

int parse_email_command(
    const char *line,
    Email *email,
    long sequence
){
    char buffer[MAX_LINE_LENGTH];
    char *fields[3];
    char *token;
    char *rest;
    int field_count = 0;

    strcpy(buffer, line);

    rest = buffer + 6;

    token = strtok(rest, ",");

    while (token != NULL && field_count < 3) {
        fields[field_count++] = token;
        token = strtok(NULL, ",");
    }

    if (field_count != 3) {
        return 0;
    }

    strncpy(
        email->sender,
        fields[0],
        sizeof(email->sender) - 1
    );

    email->sender[sizeof(email->sender) - 1] = '\0';

    strncpy(
        email->subject,
        fields[1],
        sizeof(email->subject) - 1
    );

```

```
email->subject[sizeof(email->subject) - 1] = '\0';
```

```
if (sscanf(
    fields[2],
    "%d-%d-%d",
    &email->month,
    &email->day,
    &email->year
) != 3) {
    return 0;
}
```

```
email->sequence = sequence;
```

```
return 1;
```

```
}
```

```
void process_file(FILE *input) {
    MaxHeap heap;
    char line[MAX_LINE_LENGTH];
    long sequence = 0;
```

```
    heap_init(&heap);
```

```
    while (fgets(line, sizeof(line), input) != NULL) {
        line[strcspn(line, "\r\n")] = '\0';
```

```
        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email email;
```

```
            if (parse_email_command(
                line,
                &email,
                sequence
            )) {
                heap_insert(&heap, email);
                sequence++;
            }
        }
```

```
    }
    else if (strcmp(line, "NEXT") == 0) {
        Email *next = heap_peek(&heap);
```

```
        if (next != NULL) {
```

```

        print_email(next);
    }
}
else if (strcmp(line, "READ") == 0) {
    Email removed;
    heap_remove_max(&heap, &removed);
}
else if (strcmp(line, "COUNT") == 0) {
    printf(
        "There are %d emails to read.\n",
        heap.size
    );
}
}

heap_destroy(&heap);
}

int main(int argc, char *argv[]) {
    FILE *input;

    if (argc != 2) {
        fprintf(
            stderr,
            "Usage: %s <test_file>\n",
            argv[0]
        );
        return EXIT_FAILURE;
    }

    input = fopen(argv[1], "r");

    if (input == NULL) {
        perror("Unable to open input file");
        return EXIT_FAILURE;
    }

    process_file(input);

    fclose(input);

    return EXIT_SUCCESS;
}

```

4. Generated Code — GenAI #2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_LINE_LEN    512
#define MAX_CATEGORY_LEN 32
#define MAX_SUBJECT_LEN 256
#define MAX_DATE_LEN    16
#define INITIAL_CAPACITY 16

typedef struct {
    char category[MAX_CATEGORY_LEN];
    char subject[MAX_SUBJECT_LEN];
    char date[MAX_DATE_LEN];
    long dateSortKey;
} Email;

typedef struct {
    Email *items;
    int size;
    int capacity;
} MaxHeap;

int categoryRank(const char *category) {
    if (strcmp(category, "Boss") == 0)      return 0;
    if (strcmp(category, "Subordinate") == 0) return 1;
    if (strcmp(category, "Peer") == 0)      return 2;
    if (strcmp(category, "ImportantPerson") == 0) return 3;
    if (strcmp(category, "OtherPerson") == 0) return 4;
    return 5;
}

long parseDateToSortKey(const char *date) {
    int month = 0, day = 0, year = 0;
    sscanf(date, "%d-%d-%d", &month, &day, &year);
    return (long) year * 10000L + (long) month * 100L + (long) day;
}

int isHigherPriority(const Email *a, const Email *b) {
    int rankA = categoryRank(a->category);
    int rankB = categoryRank(b->category);
```

```

    if (rankA != rankB) {
        return rankA < rankB;
    }
    return a->dateSortKey > b->dateSortKey;
}

void heapInit(MaxHeap *heap) {
    heap->items = (Email *) malloc(sizeof(Email) * INITIAL_CAPACITY);
    heap->size = 0;
    heap->capacity = INITIAL_CAPACITY;
}

void heapFree(MaxHeap *heap) {
    free(heap->items);
    heap->items = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

void heapEnsureCapacity(MaxHeap *heap) {
    if (heap->size == heap->capacity) {
        heap->capacity *= 2;
        heap->items = (Email *) realloc(heap->items, sizeof(Email) * heap->capacity);
    }
}

void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void siftUp(MaxHeap *heap, int i) {
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (isHigherPriority(&heap->items[i], &heap->items[parent])) {
            swapEmails(&heap->items[i], &heap->items[parent]);
            i = parent;
        } else {
            break;
        }
    }
}

```

```

void siftDown(MaxHeap *heap, int i) {
    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int best = i;
        if (left < heap->size && isHigherPriority(&heap->items[left], &heap->items[best])) {
            best = left;
        }
        if (right < heap->size && isHigherPriority(&heap->items[right], &heap->items[best])) {
            best = right;
        }
        if (best == i) {
            break;
        }
        swapEmails(&heap->items[i], &heap->items[best]);
        i = best;
    }
}

```

```

void heapPush(MaxHeap *heap, Email email) {
    heapEnsureCapacity(heap);
    heap->items[heap->size] = email;
    siftUp(heap, heap->size);
    heap->size++;
}

```

```

Email *heapPeek(MaxHeap *heap) {
    if (heap->size == 0) {
        return NULL;
    }
    return &heap->items[0];
}

```

```

void heapPop(MaxHeap *heap) {
    if (heap->size == 0) {
        return;
    }
    heap->items[0] = heap->items[heap->size - 1];
    heap->size--;
    if (heap->size > 0) {
        siftDown(heap, 0);
    }
}

```

```

void trimNewline(char *line) {
    size_t len = strlen(line);
    while (len > 0 && (line[len - 1] == '\n' || line[len - 1] == '\r')) {
        line[len - 1] = '\0';
        len--;
    }
}

void parseEmailLine(const char *rest, Email *outEmail) {
    char buffer[MAX_LINE_LEN];
    strncpy(buffer, rest, MAX_LINE_LEN - 1);
    buffer[MAX_LINE_LEN - 1] = '\0';

    char *categoryTok = strtok(buffer, ",");
    char *subjectTok = strtok(NULL, ",");
    char *dateTok = strtok(NULL, ",");

    strncpy(outEmail->category, categoryTok ? categoryTok : "", MAX_CATEGORY_LEN - 1);
    outEmail->category[MAX_CATEGORY_LEN - 1] = '\0';

    strncpy(outEmail->subject, subjectTok ? subjectTok : "", MAX_SUBJECT_LEN - 1);
    outEmail->subject[MAX_SUBJECT_LEN - 1] = '\0';

    strncpy(outEmail->date, dateTok ? dateTok : "", MAX_DATE_LEN - 1);
    outEmail->date[MAX_DATE_LEN - 1] = '\0';

    outEmail->dateSortKey = parseDateToSortKey(outEmail->date);
}

void handleNext(MaxHeap *heap) {
    Email *top = heapPeek(heap);
    if (top == NULL) {
        printf("No emails to read.\n");
        return;
    }
    printf("Next email:\n");
    printf("Sender: %s\n", top->category);
    printf("Subject: %s\n", top->subject);
    printf("Date: %s\n", top->date);
}

void handleRead(MaxHeap *heap) {
    heapPop(heap);
}

```

```

void handleCount(MaxHeap *heap) {
    printf("There are %d emails to read.\n", heap->size);
}

int main(int argc, char *argv[]) {
    MaxHeap heap;
    heapInit(&heap);

    FILE *input = stdin;
    if (argc > 1) {
        input = fopen(argv[1], "r");
        if (input == NULL) {
            fprintf(stderr, "Error: could not open file '%s'\n", argv[1]);
            heapFree(&heap);
            return 1;
        }
    }

    char line[MAX_LINE_LEN];
    while (fgets(line, sizeof(line), input) != NULL) {
        trimNewline(line);

        if (line[0] == '\0') {
            continue;
        }

        if (strncmp(line, "EMAIL ", 6) == 0) {
            Email newEmail;
            parseEmailLine(line + 6, &newEmail);
            heapPush(&heap, newEmail);
        } else if (strcmp(line, "NEXT") == 0) {
            handleNext(&heap);
        } else if (strcmp(line, "READ") == 0) {
            handleRead(&heap);
        } else if (strcmp(line, "COUNT") == 0) {

```

5. Code Analysis

Correctness:

ChatGPT:

- The program correctly implements a list-based binary MaxHeap using a dynamically allocated array stored in the MaxHeap structure. It does not use a pre-existing heap library.

Claude:

- This program also correctly implements a list-based binary MaxHeap with a dynamically growing array and no heap library.

Execution Time:

For a binary MaxHeap:

Insert: $O(\log n)$ worst case.

NEXT/peek: $O(1)$.

READ/remove maximum: $O(\log n)$.

COUNT: $O(1)$.

Neither implementation accidentally performs an $O(n)$ linear scan for NEXT, READ, or insertion. Priority comparisons are constant-time because sender categories and dates are represented with fixed-size fields.

Space Complexity:

ChatGPT:

- Uses a dynamically allocated Email array that begins with capacity 10 and doubles when full.
- Each email stores the sender, subject, separate month/day/year integers, and a sequence number.
- Parsing uses a local buffer[MAX_LINE_LENGTH] and a small array of three pointers. These temporary objects are fixed-size, so they use $O(1)$ auxiliary space.

Claude:

- Uses a dynamically allocated Email array that begins with capacity 16 and doubles when full.
- Each email stores category, subject, the original date string, and a precomputed numeric date key.
- parseEmailLine() uses a fixed-size local parsing buffer, which is $O(1)$ auxiliary memory.

Maintainability:

ChatGPT:

The code is organized into small functions for priority comparison, heap initialization/destruction, resizing, heapify operations, insertion, peek, removal, printing, parsing, file processing, and main. The snake_case naming style is consistent. Comments explain the purpose of most functions without making the program difficult to follow. Parsing, heap operations, and command processing are separated reasonably well.

Claude:

This version is also well organized. It clearly separates helper functions, MaxHeap operations, input parsing, command handlers, and main. Function names such as categoryRank, isHigherPriority, siftUp, siftDown, handleNext, and handleRead describe their jobs clearly. Constants are provided for line length, category length, subject length, date length, and initial capacity.

6. Selection

I selected Claude as the basis for the final implementation.

Reason:

I selected Claude because it combines correct heap behavior with stronger error handling and more controlled input validation. It checks the result of both malloc() and realloc(), while ChatGPT does not. Its parse_email_command() also rejects a command if it does not contain exactly three comma-separated fields or if the date cannot be parsed into three integers.

7. Improvements Made

Correctness:

The final implementation improves correctness and robustness by validating the three required fields in an EMAIL command and checking that the date contains three numeric components before inserting the email.

Execution Time:

The final implementation keeps all required heap operations at the expected complexities: $O(\log n)$ insertion, $O(1)$ NEXT, $O(\log n)$ READ, and $O(1)$ COUNT. It does not use a linear search to find the highest-priority email.

Space Complexity:

The final implementation continues to use $O(n)$ heap storage and $O(1)$ auxiliary working memory. The dynamically allocated list grows only when needed and is released with `heap_destroy()` at the end of processing. Fixed-size parsing buffers and one temporary Email used during swaps do not change the $O(n)$ total space complexity.

Maintainability:

The final implementation uses separate functions for sender priority, date comparison, heap initialization/destruction, growth, heapify-up, heapify-down, insertion, peek, removal, output formatting, command parsing, file processing, and main. This makes each part of the program easier to test or change independently.